

README.md

DriveCommand – Developer Documentation

DriveCommand is a multi-tenant SaaS fleet management platform for trucking operators. It provides three separate portals: an Owner portal for fleet owners/managers, a Driver portal for drivers, and a SysAdmin portal for the DriveCommand team.

Tech Stack

Layer	Technology
Framework	Next.js 15 (App Router)
Language	TypeScript
Database	PostgreSQL via Supabase
ORM	Prisma 7 with @prisma/adapters-pg
Styling	Tailwind CSS + shadcn/ui
Auth	Custom AES-256-GCM session cookie
Email	Gmail SMTP via Nodemailer
File Storage	Cloudflare R2 / AWS S3
AI	Anthropic Claude (document reading, profit predictor)
Maps	Leaflet + react-leaflet, Google Maps (autocomplete)
Deployment	Vercel

Portal Map

DriveCommand routes users into one of three portals based on their role and session state.

Owner Portal

Fleet owners and managers. URL prefix: `/`

Pages: `/dashboard`, `/trucks`, `/drivers`, `/routes`, `/loads`,
`/invoices`, `/payroll`, `/crm`, `/compliance`, `/ai-documents`,
`/profit-predictor`, `/lane-analytics`, `/ifta`, `/live-map`, `/fuel`,
`/safety`, `/tags`, `/settings`, `/support`

Driver Portal

Drivers. URL prefix: ``/my-*`, `/hours`, `/incidents`, `/messages``

Pages: ``/my-route`, `/my-load`, `/my-tickets`, `/hours`, `/incidents`, `/messages``

SysAdmin Portal

DriveCommand internal staff. URL prefix: ``/admin*`, `/tenants``

Pages: ``/admin-dashboard`, `/admin-support`, `/tenants`, `/admin``

Table of Contents

- [Architecture](./architecture.md) - System design, portal routing, multi-tenancy, RLS, middleware flow
- [Authentication](./auth.md) - Session management, role model, auth helpers, guard patterns
- [Database](./database.md) - Schema reference, Prisma setup, RLS policies, `bypass_rls`, migrations
- [Tech Stack](./stack.md) - Dependency versions, library rationale, environment variables
- [Modules](./modules.md) - Feature modules (CRM, dispatch, finance, compliance, AI, integrations)
- [Local Setup](./setup.md) - Prerequisites, environment variables, database setup, running locally
- [Deployment](./deployment.md) - Vercel deployment, build command, environment variables, cron jobs
- [Email](./email.md) - Gmail SMTP setup, Nodemailer client, email templates, notification log, idempotency

Quick Start

See [Local Setup](./setup.md) to get the app running locally.

Architecture.md

Architecture

This document describes DriveCommand's system design: how its three portals are structured, how tenants are isolated, how the middleware enforces access control, and how a typical request flows from browser to database.

System Overview

DriveCommand is a Next.js App Router application with three distinct user portals sharing a single PostgreSQL database. Each portal is implemented as a route group under ``src/app/``:

Route Group	Portal	Users
<code>`(owner)/`</code>	Owner portal	Fleet owners and managers
<code>`(driver)/`</code>	Driver portal	Truck drivers
<code>`(admin)/`</code>	SysAdmin portal	DriveCommand internal staff

All three portals share the same Prisma client, the same session cookie, and the same PostgreSQL database. Isolation between fleet operators is enforced at the database level with Row Level Security (RLS).

Multi-Tenancy Design

Every fleet operator is a ****Tenant**** – one row in the ``Tenant`` table. All data entities (trucks, routes, drivers, loads, etc.) carry a ``tenantId`` `UUID`` foreign key referencing ``Tenant.id``.

How tenant isolation is enforced

1. ****Row Level Security (RLS)**** is enforced at the PostgreSQL level via Supabase RLS policies. Every tenant-scoped table has a policy that checks ``current_setting('app.tenant_id') = "tenantId"::text`` on all operations (SELECT, INSERT, UPDATE, DELETE).

2. ****`app.tenant\_id` session variable**** is set at the start of each API route or server action via:

```
```sql
SELECT set_config('app.tenant_id', '<tenantId>', TRUE)
```
```

The ``TRUE`` parameter scopes this setting to the current transaction only.

3. ****Middleware injects `x-tenant-id`**** as a request header from the validated session cookie. Server actions and API routes read this header to get the tenant ID without calling ``getSession()`` again.

`bypass_rls` pattern

Some operations must bypass RLS because they run before a tenant context exists (login) or need to span tenants (sysadmin):

```
```typescript
// Set bypass_rls=on as the first statement in a transaction:
const [, user] = await prisma.$transaction([
 prisma.$executeRaw`SELECT set_config('app.bypass_rls', 'on', TRUE)` ,
 prisma.user.findUnique({ where: { id: session.userId } }),
]);
```
```

The RLS policy checks ``current_setting('app.bypass_rls', TRUE) = 'on'`` and permits access if true. The ``TRUE`` parameter ensures the bypass flag is scoped to the current transaction and does not leak across requests.

For ``$queryRaw`` calls (outside a transaction), RLS is bypassed entirely because ``$queryRaw`` runs as the database superuser.

`Middleware Flow`

The middleware (``src/middleware.ts``) runs on every request except static assets. It enforces authentication, tenant context, and role-based routing.

****5-step flow:****

1. ****Public paths pass through.**** Requests to `/sign-in`, `/sign-up`, `/api/auth/*`, `/api/warmup`, `/api/webhooks`, `/track`, `/accept-invitation`, and static files (`/_next/static`, `/_next/image`, `/favicon.*`) skip all checks.
2. ****Unauthenticated requests are redirected.**** If no valid `session` cookie exists (or decryption fails), the user is redirected to `/sign-in?redirect_url=<original path>`.
3. ****Authenticated users with no `tenantId` are redirected to `/onboarding`.** A session without a `tenantId` means the user exists but has not been assigned to a tenant yet (e.g., sysadmin in a broken state, or an edge case during tenant creation). API paths and `/onboarding` itself are excluded from this redirect.
4. ****System admins are restricted to the admin portal.**** If `session.isSystemAdmin` is true, the user may only access paths starting with `/admin`, `/admin-support`, `/admin-dashboard`, `/tenants`, `/unauthorized`, `/onboarding`, or `/api`. Any other path redirects to `/admin-support`.
5. ****DRIVER role users are redirected away from owner paths.**** If the session role is `DRIVER` and the requested path matches the `OWNER_PATHS` list (`/dashboard`, `/trucks`, `/drivers`, `/routes`, `/loads`, `/invoices`, `/payroll`, `/crm`, `/settings`, `/compliance`, `/ai-documents`, `/profit-predictor`, `/lane-analytics`, `/ifta`, `/live-map`, `/fuel`, `/safety`, `/tags`), the user is redirected to `/my-route`.
6. ****Authenticated tenant users get `x-tenant-id` injected.**** All authenticated requests that pass the above checks have `x-tenant-id: <tenantId>` added as a request header before being forwarded to the Next.js server.

Request Lifecycle

A typical owner portal page request flows:

...

Browser

- Middleware (validates session cookie, injects x-tenant-id header)
- Next.js Server Component (calls getSession() for role/userId)
- Server Action or API Route
 - (reads x-tenant-id header, calls set_config to set tenant context)
 - (runs Prisma query through RLS-filtered PostgreSQL)
- Returns data to Server Component
- Renders HTML to client

...

****Key points:****

- Server components call `getSession()` (cached per request with `React.cache()`) to read role and userId from the session cookie.
- Server actions call `requireAuth()` and `requireRole()` at the top of each function as defense-in-depth guards, even though middleware already enforces role routing.
- Prisma queries run through RLS — no explicit `WHERE tenantId = ?` is needed in most queries because RLS filters rows automatically once `app.tenant_id` is set.

File Structure Overview

...

src/

 app/

- (owner)/ # Owner portal pages and server actions
- (driver)/ # Driver portal pages and server actions
- (admin)/ # SysAdmin portal pages
- api/ # API routes (auth, cron, GPS, documents, webhooks,

etc.)

 lib/

- auth/ # session.ts, server.ts, roles.ts
- db/ # prisma.ts (singleton Prisma client + connection pool)
- email/ # Resend client, email sender functions, templates


```
notifications/ # Notification log utilities (idempotent email
tracking)
storage/      # S3/R2 file upload helpers (presigned URLs, multipart)
finance/      # Revenue/expense calculation helpers (Decimal.js)
safety/       # Safety event scoring utilities
geo/          # Geofencing utilities (bounding-box state detection)
prisma/
  schema.prisma # Single schema file — all models and enums
scripts/
  migrate.mjs   # Raw SQL migration runner (used in Vercel build
command)
docs/          # Developer documentation (this directory)
```
```

---

## ## Key Design Decisions

**\*\*Next.js App Router server components\*\*** are used for initial data fetching. No client-side fetch on first load — data arrives already rendered. Client components are used only for interactivity (forms, modals, live maps, polling).

**\*\*Prisma with `@prisma/adapters-pg`\*\*** uses a singleton `pg.Pool` (max 5 connections) stored on `globalThis`. This survives Vercel's serverless module-level warm reuse between invocations, avoiding a new TCP handshake to Supabase on every request.

**\*\*`DATABASE\_URL` must use Supabase's Session Mode pooler\*\*** on port **\*\*6543\*\*** (not 5432). Transaction mode drops connections after each transaction, which defeats pooling and causes `app.tenant\_id` config to be lost.

**\*\*Custom AES-256-GCM session cookie\*\*** uses the Web Crypto API, which works in both Edge Runtime (middleware) and Node.js (server components). No third-party auth library is used — the implementation is in `src/lib/auth/session.ts` and is fully transparent.

**\*\*No Prisma migrations in production.\*\*** Schema changes are applied via `scripts/migrate.mjs`, which runs raw SQL migration files from a

```
`migrations/` directory as part of the Vercel build command: `node
scripts/migrate.mjs && prisma generate && next build`.
```

**\*\*Prisma client output\*\*** is `src/generated/prisma/` (not the default  
`node\_modules`). This is required for Vercel deployment compatibility.

# Authentication.md

## # Authentication

DriveCommand uses a custom cookie-based session system. There is no NextAuth, no JWT library, and no third-party auth provider. The session is encrypted with AES-256-GCM using the Web Crypto API, stored as an ``httpOnly`` cookie named ``session``, and expires after 7 days.

### \*\*Key files:\*\*

- ``src/lib/auth/session.ts`` - ``encrypt``, ``decrypt``, ``getSession``, ``setSession``, ``clearSession``
- ``src/lib/auth/server.ts`` - ``requireAuth``, ``requireRole``, ``isSystemAdmin``, ``getCurrentUser``
- ``src/middleware.ts`` - request-level auth guard and ``x-tenant-id`` injection

---

## ## SessionData Shape

```
```typescript
interface SessionData {
  userId: string;          // UUID - User.id in the database
  email: string;
  role: string;            // 'OWNER' | 'MANAGER' | 'DRIVER'
  tenantId: string;        // UUID - Tenant.id in the database
  firstName?: string;
  lastName?: string;
  isSystemAdmin?: boolean;
}
```
```

``tenantId`` is the primary scope for all data access. Every server action reads ``session.tenantId`` to construct tenant-scoped database queries.

``isSystemAdmin`` in the session is supplementary - it is set at login time for convenience (to avoid a DB call on every middleware check). The authoritative check is via ``isSystemAdmin()`` in ``server.ts``, which hits the database directly.

---

## ## Token Format

The cookie value is three base64url-encoded segments joined by `:`:

```

base64url(iv) : base64url(authTag) : base64url(ciphertext)

```

- **IV**: 12 bytes, randomly generated per `encrypt()` call using `crypto.getRandomValues()`
- **Auth tag**: 16 bytes, produced by AES-GCM and appended to ciphertext by Web Crypto
- **Ciphertext**: the AES-256-GCM encrypted JSON of the `SessionData` object

The `AUTH_SECRET` environment variable is SHA-256 hashed to derive the 256-bit AES key. This means `AUTH_SECRET` can be any string of 32+ characters – the hash normalizes it to the required 32-byte key length.

Tampered tokens (wrong IV, wrong auth tag, or wrong key) cause AES-GCM decryption to throw, which `decrypt()` catches and returns `null`.

---

## ## Session Functions

All session functions are in `src/lib/auth/session.ts`.

**`encrypt(payload: SessionData): Promise<string>`**

Encrypts the payload with AES-256-GCM using a fresh random IV. Returns the `iv:authTag:ciphertext` token string.

**`decrypt(token: string | undefined): Promise<SessionData | null>`**

Decrypts the token. Returns `null` on any failure – tampered token, wrong key, missing token, or malformed format. Never throws.

**`getSession(): Promise<SessionData | null>`**

Reads and decrypts the ``session`` cookie from ``next/headers``. Wrapped with ``React.cache()`` so decryption runs at most once per request – all callers within the same request share the cached result. Use this in server components, server actions, and API routes.

**`**`setSession(data: SessionData): Promise<void>`**`**

Encrypts and sets the ``session`` cookie with these attributes:

- ``httpOnly: true`` – inaccessible to JavaScript
- ``secure: true`` in production – HTTPS only
- ``sameSite: 'lax'`` – protects against CSRF while allowing navigation links
- ``maxAge: 604800`` (7 days)
- ``path: '/'`` – available on all routes

**`**`clearSession(): Promise<void>`**`**

Deletes the session cookie. Called on logout.

---

## **## Auth Helpers (server.ts)**

All helpers are in ``src/lib/auth/server.ts``. They call ``getSession()`` internally (benefiting from the ``React.cache()`` deduplication).

**`**`getRole(): Promise<UserRole | null>`**`**

Reads the role from the session cookie. No database call. Use for role checks in server actions where a DB roundtrip is unacceptable.

**`**`requireAuth(): Promise<string>`**`**

Throws ``Error('Unauthorized: Authentication required')`` if there is no session. Returns ``session.userId`` (a UUID string) on success. Call this at the top of every protected server action.

**`**`requireRole(allowedRoles: UserRole[]): Promise<UserRole>`**`**

Throws if the session role is not in ``allowedRoles``. Returns the matched role. Use for role-gated server actions.

**`**`isSystemAdmin(): Promise<boolean>`**`**

Runs a ``$queryRaw`` SELECT to check ``User.isSystemAdmin`` in the database. Returns false if not authenticated. **\*\*This makes a database call – use sparingly.\*\*** For routine role checks, use ``getRole()`` or ``requireRole()``.

### **\*\*`getCurrentUser()`\*\***

Fetches the User record using a ``bypass_qls`` transaction (see `[Database](../database.md)` for the pattern). Returns ``null`` if not authenticated. **\*\*This makes a database call – use sparingly.\*\***

---

## **## Role Model**

Three roles exist in the ``UserRole`` enum:

| Role                   | Description                                                                                                                                                                                               |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>`OWNER`</code>   | Full access to all owner portal features for their tenant                                                                                                                                                 |
| <code>`MANAGER`</code> | Same access as OWNER (not yet differentiated in middleware)                                                                                                                                               |
| <code>`DRIVER`</code>  | Restricted to driver portal paths only ( <code>`/my-route`</code> , <code>`/my-load`</code> , <code>`/my-tickets`</code> , <code>`/hours`</code> , <code>`/incidents`</code> , <code>`/messages`</code> ) |

Additionally, ``User.isSystemAdmin: boolean`` flags DriveCommand internal staff. System admins have a ``tenantId`` (they belong to a tenant) but are routed exclusively to admin portal paths (``/admin*``, ``/tenants``). Their ``tenantId`` is irrelevant for data access – sysadmin operations bypass RLS entirely.

---

## **## Login and Logout Flow**

### **\*\*Login\*\*** – ``POST /api/auth/login``

1. Reads ``email`` and ``password`` from the request body.
2. Finds the ``User`` record by email using a ``bypass_qls`` transaction (no tenant context at login time).
3. Compares the submitted password against ``User.passwordHash`` with ``bcrypt.compare()``.

```
4. Calls `setSession()` with the user's `userId`, `email`, `role`,
`tenantId`, `firstName`, `lastName`, and `isSystemAdmin`.
5. Returns a redirect response to the appropriate portal (`/dashboard` for
owners/managers, `/my-route` for drivers, `/admin-support` for sysadmins).
```

**\*\*Logout\*\*** - `POST /api/auth/logout`

```
1. Calls `clearSession()` to delete the session cookie.
2. Redirects to `/sign-in`.
```

**\*\*Accept Invitation\*\*** - `GET /accept-invitation?token=<id>` → `POST  
/api/auth/accept-invitation`

```
1. Validates the `DriverInvitation` record by UUID `id` and checks `status
=== 'PENDING'` and `expiresAt > now`.
2. Creates a `User` record with the invitation's `email`, `firstName`,
`lastName`, `tenantId`, and `role`.
3. Sets `invitation.status = 'ACCEPTED'`.
4. Calls `setSession()` to auto-log-in the new user.
5. Redirects to `/my-route` (drivers) or `/dashboard` (owners).
```

---

## ## Server Action Pattern

Every protected server action starts with these guards:

```
```typescript  
// At the top of every protected server action:  
const userId = await requireAuth();  
const role = await requireRole([UserRole.OWNER, UserRole.MANAGER]);  
const session = await getSession();  
// session.tenantId is the tenant scope for this request  
```
```

Because `getSession()` is wrapped with `React.cache()`, calling it after  
`requireAuth()` and `requireRole()` (which also call `getSession()`  
internally) does not result in additional cookie reads or decryption work.

---

## ## isSystemAdmin Bypass



System admin status is checked via ``$queryRaw``, which bypasses RLS entirely (runs as the database superuser):

```
```typescript
const rows = await prisma.$queryRaw<{ isSystemAdmin: boolean }[]>`
  SELECT "isSystemAdmin" FROM "User" WHERE id = ${session.userId}::uuid
LIMIT 1
`;
```
```

The ``isSystemAdmin`` field on the session cookie is populated at login time for convenience — it allows the middleware to redirect admin users without a database call. However, it is not the sole authority. Any server action that needs to verify admin status should call ``isSystemAdmin()`` from ``server.ts``, which hits the database.

---

## ## Admin Portal Authentication

The SysAdmin portal (``/admin*``) uses a separate authentication mechanism: a plain ``ADMIN_SECRET_KEY`` environment variable. Logging in at ``/admin/login`` compares the submitted password against ``ADMIN_SECRET_KEY`` with a 500ms artificial delay (brute-force resistance). On success, a separate ``admin_session`` cookie is set (encrypted with the same AES-256-GCM system).

The middleware checks the ``admin_session`` cookie for admin portal paths and bypasses all tenant session checks for those routes.

Database.md

## # Database

DriveCommand uses PostgreSQL hosted on Supabase. The schema is managed with Prisma. All tenant-scoped tables use Row Level Security (RLS) enforced at the Supabase level.

### \*\*Key files:\*\*

- `prisma/schema.prisma` - single schema file, all models and enums
- `src/lib/db/prisma.ts` - singleton Prisma client and connection pool
- `src/generated/prisma/` - generated Prisma client output
- `scripts/migrate.mjs` - raw SQL migration runner

---

## ## Prisma Setup

The Prisma client is configured in `src/lib/db/prisma.ts` using the `@prisma/adaptor-pg` driver adapter with a `pg.Pool` singleton.

### \*\*Connection pool:\*\*

- `max: 5` connections per Vercel worker instance
- Stored on `globalThis` so the pool persists across Vercel's serverless module-level warm reuse between invocations
- Without `globalThis`, each cold start would open a new TCP connection to Supabase

### \*\*`DATABASE\_URL` must use Supabase's Session Mode pooler on port 6543.\*\*

Do not use port 5432 (Transaction Mode). Transaction mode drops connections after each transaction, which defeats pooling and causes `set\_config('app.tenant\_id', ...)` session variables to be lost.

...

```
postgresql://postgres.[ref]:[password]@aws-0-[region].pooler.supabase.com:6543/postgres
```

...

### \*\*Transaction options (`TX\_OPTIONS`):\*\*

```
```typescript
```

```
export const TX_OPTIONS = { maxWait: 15000, timeout: 30000 }
```

```
These are raised from Prisma's defaults (maxWait: 2000, timeout: 5000) to handle burst concurrency under parallel page renders.
```

****Client generation output:****

The Prisma client is generated to ``src/generated/prisma/`` (not the default ``node_modules``). This is required for Vercel deployment. After any schema change, run:

```
```bash
npx prisma generate
```
```

Schema Overview – Models

The schema has 29 models covering all platform features.

| Model | Purpose | Key Fields |
|---------------------------------|--|--|
| --- | --- | --- |
| <code>`Tenant`</code> | Fleet operator account | <code>`id`</code> , <code>`name`</code> , <code>`slug`</code> , <code>`isActive`</code> , <code>`profitMarginThreshold`</code> |
| <code>`User`</code> | Owner, Manager, or Driver | <code>`id`</code> , <code>`tenantId`</code> , <code>`email`</code> , <code>`passwordHash`</code> , <code>`role`</code> , <code>`isSystemAdmin`</code> , <code>`isActive`</code> |
| <code>`Truck`</code> | Vehicle in the fleet | <code>`id`</code> , <code>`tenantId`</code> , <code>`make`</code> , <code>`model`</code> , <code>`year`</code> , <code>`vin`</code> , <code>`licensePlate`</code> , <code>`odometer`</code> , <code>`documentMetadata`</code> (JSONB) |
| <code>`DriverInvitation`</code> | Email invite for new drivers or owners | <code>`id`</code> , <code>`tenantId`</code> , <code>`email`</code> , <code>`role`</code> (defaults DRIVER), <code>`status`</code> (PENDING/ACCEPTED/EXPIRED/CANCELLED), <code>`expiresAt`</code> |
| <code>`Route`</code> | Assigned trip with driver + truck | <code>`id`</code> , <code>`tenantId`</code> , <code>`driverId`</code> , <code>`truckId`</code> , <code>`origin`</code> , <code>`destination`</code> , <code>`status`</code> (PLANNED/IN_PROGRESS/COMPLETED), <code>`version`</code> (optimistic locking) |
| <code>`RouteStop`</code> | Multi-stop waypoints on a route | <code>`id`</code> , <code>`routeId`</code> , <code>`tenantId`</code> , <code>`position`</code> (1-based), <code>`type`</code> (PICKUP/DELIVERY), <code>`status`</code> (PENDING/ARRIVED/DEPARTED), <code>`geofenceHit`</code> |
| <code>`Document`</code> | Uploaded file metadata | <code>`id`</code> , <code>`tenantId`</code> , <code>`truckId?`</code> , <code>`routeId?`</code> , <code>`driverId?`</code> , <code>`s3Key`</code> , <code>`documentType?`</code> , <code>`expiryDate?`</code> |
| <code>`MaintenanceEvent`</code> | Completed service record | <code>`id`</code> , <code>`tenantId`</code> , <code>`truckId`</code> , <code>`serviceType`</code> , <code>`serviceDate`</code> , <code>`odometerAtService`</code> , <code>`cost`</code> |

| `ScheduledService` | Upcoming service by interval | `id`, `tenantId`,
`truckId`, `intervalDays?`, `intervalMiles?`, `isCompleted` |
| `GPSLocation` | Truck GPS ping | `id`, `tenantId`, `truckId`,
`latitude`, `longitude`, `speed`, `timestamp` |
| `SafetyEvent` | Harsh driving event from ELD/telematics | `id`,
`tenantId`, `truckId`, `driverId?`, `eventType`, `severity`, `gForce?`,
`speed?`, `timestamp` |
| `FuelRecord` | Fuel fill-up record | `id`, `tenantId`, `truckId`,
`fuelType`, `quantity`, `totalCost?`, `odometer`, `timestamp` |
| `NotificationLog` | Email send audit log | `id`, `tenantId`,
`idempotencyKey` (unique), `notificationType`, `status`
(PENDING/SENT/FAILED) |
| `Tag` | Color label for trucks/drivers | `id`, `tenantId`, `name`,
`color` |
| `TagAssignment` | Tag applied to a Truck or User | `id`, `tenantId`,
`tagId`, `truckId?`, `userId?` |
| `ExpenseCategory` | Named expense bucket | `id`, `tenantId`, `name`,
`isSystemDefault` |
| `RouteExpense` | Expense on a route (soft delete) | `id`, `tenantId`,
`routeId`, `categoryId`, `amount`, `deletedAt?` |
| `ExpenseTemplate` | Reusable expense preset | `id`, `tenantId`, `name` |
| `ExpenseTemplateItem` | Line item in an expense template | `id`,
`templateId`, `categoryId`, `tenantId`, `amount`, `description` |
| `RoutePayment` | Revenue received for a route (soft delete) | `id`,
`tenantId`, `routeId`, `amount`, `status` (PENDING/PAID), `paidAt?` |
| `Customer` | CRM contact / shipper | `id`, `tenantId`, `companyName`,
`priority`, `status`, `totalRevenue`, `emailNotifications` |
| `CustomerInteraction` | CRM activity log | `id`, `tenantId`,
`customerId`, `type`, `subject`, `isAutomated` |
| `Invoice` | Invoice to a customer | `id`, `tenantId`, `customerId?`,
`invoiceNumber`, `amount`, `status` (DRAFT/SENT/PAID/OVERDUE/CANCELLED) |
| `InvoiceItem` | Line item on an invoice | `id`, `invoiceId`, `tenantId`,
`description`, `quantity`, `unitPrice`, `amount` |
| `PayrollRecord` | Driver pay period record | `id`, `tenantId`,
`driverId`, `periodStart`, `periodEnd`, `totalPay`, `status`
(DRAFT/APPROVED/PAID) |
| `Load` | Dispatched load/shipment | `id`, `tenantId`, `loadNumber`,
`customerId`, `routeId?`, `status`
(PENDING→DISPATCHED→PICKED_UP→IN_TRANSIT→DELIVERED→INVOICED),
`trackingToken?` |

```
| `TenantIntegration` | Third-party integration config | `id`, `tenantId`,
`provider`, `category`, `enabled`, `configJson` (JSONB) |
| `SupportTicket` | Help ticket from tenant owner or driver | `id`,
`ticketNumber` (unique, TKT-NNNN), `tenantId`, `category`, `priority`,
`status` |
| `TicketMessage` | Message thread on a support ticket | `id`, `ticketId`,
`senderType` (OWNER/ADMIN), `body` |
```

Row Level Security (RLS)

All tenant-scoped tables have RLS enabled in Supabase. The enforcement pattern:

1. A Supabase RLS policy is applied to the table:

```
```sql
CREATE POLICY tenant_isolation_policy ON "Truck"
USING (
 current_setting('app.bypass_rls', TRUE) = 'on'
 OR "tenantId"::text = current_setting('app.tenant_id', TRUE)
);
```
```

2. At the start of each API route or server action, the tenant ID is set:

```
```typescript
await prisma.$executeRaw`SELECT set_config('app.tenant_id',
${tenantId}, TRUE)`;
```
```

The `TRUE` parameter scopes this setting to the current transaction.

3. All subsequent Prisma queries on the connection run through the RLS filter automatically. An explicit `WHERE tenantId = ?` is still included in most queries as defense-in-depth, but RLS ensures that even if a query omits the filter, no cross-tenant data leaks.

Tables without RLS:

- `SupportTicket` and `TicketMessage` – sysadmin needs cross-tenant visibility

```
- Tenant-scoped queries on these tables use explicit `WHERE tenantId = ?`  
in server actions
```

```
---
```

bypass_rols Pattern

Some operations must access data without a tenant context filter:

```
- **Login** - no session exists yet; the `User` record must be fetched by  
email across all tenants to validate credentials  
- **Sysadmin operations** - span multiple tenants  
- **`getCurrentUser()`** - the user may need to fetch their own record  
before tenant context is set
```

```
```typescript  
// In a Prisma transaction, set bypass_rols=on before the query:
const [, user] = await prisma.$transaction([
 prisma.$executeRaw`SELECT set_config('app.bypass_rols', 'on', TRUE)`,
 prisma.user.findUnique({ where: { id: session.userId } }),
]);
```
```

The `TRUE` parameter on `set\_config` scopes the bypass flag to the current transaction. It does not leak across requests or connections.

For `\$queryRaw` calls (outside a Prisma `\$transaction`), RLS is bypassed entirely because `\$queryRaw` runs as the database superuser in Supabase. This is used in `isSystemAdmin()`:

```
```typescript  
const rows = await prisma.$queryRaw<{ isSystemAdmin: boolean }[]>`
 SELECT "isSystemAdmin" FROM "User" WHERE id = ${session.userId}::uuid
LIMIT 1
`;
```
```

```
---
```

Migrations

Prisma migrations are NOT used in production. Instead, raw SQL migration files from a ``migrations/`` directory are applied by ``scripts/migrate.mjs`` as part of the Vercel build command:

```
```\nnode scripts/migrate.mjs && prisma generate && next build\n```
```

The migration script runs each SQL file in alphabetical order inside an atomic transaction. If any migration fails, the build fails with a non-zero exit code and Vercel aborts the deployment.

**\*\*For local development\*\***, apply schema changes directly with:

```
```\nbash\nnpm run prisma db push\n```
```

This pushes the ``schema.prisma`` state directly to the database without creating migration files. Do not use ``prisma migrate dev`` – the project uses the manual migration runner pattern.

Common Query Patterns

****Standard tenant-scoped query in a server action:****

```
```\ntypescript\nconst session = await getSession();\nconst tenantId = session!.tenantId;\n\nconst [, trucks] = await prisma.$transaction([\n  prisma.$executeRaw`SELECT set_config('app.tenant_id', ${tenantId},\nTRUE)`,\n  prisma.truck.findMany({ where: { tenantId } })),\n]);\n```
```



The ``where: { tenantId }`` clause is defense-in-depth – RLS already filters by ``app.tenant_id``, but the explicit clause makes the intent clear and protects against future policy misconfiguration.

**\*\*All money amounts\*\*** use ``Decimal`` (Prisma's ``@db.Decimal(10, 2)``) and are calculated using ``Prisma.Decimal`` or ``Decimal.js`` in application code. Never convert to JavaScript ``number`` for financial calculations.

**\*\*Soft delete\*\*** is used for financial records (``RouteExpense.deletedAt``, ``RoutePayment.deletedAt``). Queries filter with ``where: { deletedAt: null }``. Hard delete is used for non-financial configuration records (expense templates, tags, loads, invoices with DRAFT status).

# Technology Stack.md

## # Technology Stack

Complete reference for every major dependency in DriveCommand, with versions and rationale.

---

### ## 1. Core Framework: Next.js 16 (App Router)

**\*\*Version:\*\*** `next@^16.1.6` · `react@^19.0.0` · `react-dom@^19.0.0`

Next.js with the App Router is the foundation of DriveCommand. All pages are React Server Components (RSC) by default. Client interactivity is opt-in via the `'use client'` directive.

**\*\*Key conventions in this project:\*\***

- **\*\*Route groups\*\*** – Three portals, each in a route group so the group name is not part of the URL:
  - ``src/app/(owner)/`` – Fleet owner portal
  - ``src/app/(driver)/`` – Driver portal
  - ``src/app/(admin)/`` – SysAdmin portal
- **\*\*API routes\*\*** – ``src/app/api/`` (REST endpoints for GPS, cron jobs, file uploads)
- **\*\*Server Actions\*\*** – defined in ``actions/`` subdirectories within each portal's route group. Called directly from Server Components or via ``useActionState`` in Client Components.
- **\*\*Layouts\*\*** – each portal has its own ``layout.tsx`` that enforces authentication and wraps all pages in that group's navigation shell.

---

### ## 2. ORM: Prisma 7.4

**\*\*Version:\*\*** `prisma@^7.4.0` · `@prisma/client@^7.4.0` ·  
`@prisma/adapters@^7.4.0`

Prisma is the database ORM. Prisma 7 requires a driver adapter for PostgreSQL connection pooling.

### **\*\*Key details:\*\***

- Uses the `pg` driver adapter (`@prisma/adapter-pg`) – required for Prisma 7 with PostgreSQL pooling via Supabase.
- Prisma Client is generated to `src/generated/prisma/` via `generator client { output = "../src/generated/prisma" }` in `prisma/schema.prisma`.
- Singleton client pattern in `src/lib/db/prisma.ts` survives Vercel warm function reuse (prevents exhausting connection limits).

### **\*\*Common commands:\*\***

```
```bash
# Apply schema changes to the database (dev only – replaces migrations)
npx prisma db push

# Regenerate the Prisma client after any schema change
npx prisma generate

# Open the Prisma data browser
npx prisma studio
```
```

---

## **## 3. Database: PostgreSQL via Supabase**

Hosted PostgreSQL managed by [Supabase] (<https://supabase.com>). Supabase provides:

- Connection via `pg.Pool` using Supabase's **\*\*Session Mode pooler\*\*** (port **\*\*6543\*\***, not 5432) – required for Vercel serverless deployments.
- Row Level Security (RLS) policies that enforce tenant isolation at the database layer. Every query runs under a `current\_tenant\_id()` function that reads the active tenant from the session context.
- Direct connection (port 5432) is fine for local development.

---

## **## 4. Styling: Tailwind CSS + shadcn/ui**

```
Versions: `tailwindcss@^3.4.1` · `class-variance-authority@^0.7.1` ·
`tailwind-merge@^3.4.1` · `clsx@^2.1.1` · `lucide-react@^0.564.0`

- **Tailwind CSS** – utility-first CSS framework. Configuration in
`tailwind.config.ts`.
- **shadcn/ui** – unstyled [Radix UI] (https://www.radix-ui.com/)
primitives styled with Tailwind. Components live in `src/components/ui/`.
Add new components with `npx shadcn@latest add <component>`.
- **Radix UI packages** used directly:
 - `@radix-ui/react-dialog`
 - `@radix-ui/react-alert-dialog`
 - `@radix-ui/react-popover`
 - `@radix-ui/react-select`
 - `@radix-ui/react-separator`
 - `@radix-ui/react-slot`
 - `@radix-ui/react-switch`
 - `@radix-ui/react-tabs`
 - `@radix-ui/react-tooltip`
- **`class-variance-authority`** – component variant patterns (e.g. Button
sizes/styles).
- **`tailwind-merge`** + **`clsx`** – conditional class composition via
the `cn()` utility in `src/lib/utils.ts`.
- **`lucide-react`** – icon library used throughout the UI.
```

---

## ## 5. Email: Nodemailer via Gmail SMTP

```
Version: `nodemailer@^8.0.1` · `@react-email/components@^1.0.7` ·
`@react-email/render@^2.0.4`
```

DriveCommand sends transactional emails using Gmail SMTP via Nodemailer.  
Email templates are written as React components and rendered to HTML using  
`@react-email/render`.

```
- **Active client:** `src/lib/email/gmail-client.ts`
- **Legacy client:** `src/lib/email/resend-client.ts` (kept for reference,
not active)
```

```
- **Required env vars:** `GMAIL_USER`, `GMAIL_APP_PASSWORD` (Google App Password – not your account password), `GMAIL_FROM_NAME` (optional display name)
```

See [Email docs](./email.md) for full setup instructions and all email trigger points.

---

## ## 6. Validation: Zod

```
Version: `zod@^4.3.6`
```

Zod is used for schema validation on all server action inputs and API request bodies. Every form submission is validated with a Zod schema before touching the database.

Example pattern:

```
```typescript
const schema = z.object({
  name: z.string().min(1),
  rate: z.string().transform(val => new Prisma.Decimal(val)),
});
const parsed = schema.parse(Object.fromEntries(formData));
```
```

---

## ## 7. File Storage: AWS S3 / Cloudflare R2

```
Version: `@aws-sdk/client-s3@^3.990.0` .
`@aws-sdk/s3-request-presigner@^3.990.0`
```

Supports both Cloudflare R2 (preferred) and AWS S3 via a configurable `S3\_ENDPOINT` env var. Used for driver document uploads (PDFs, images).

- **\*\*Presigned URLs\*\*** – the browser uploads directly to R2/S3 (not through Next.js), avoiding file size limits and unnecessary server load.
- **\*\*Multipart upload\*\*** for files larger than 5 MB – handled via presigned part URLs.

```
- Key validation enforces tenant isolation: every s3Key is prefixed with the tenant's ID.
- **Required env vars:** `S3_ENDPOINT`, `S3_REGION`, `S3_ACCESS_KEY_ID`,
 `S3_SECRET_ACCESS_KEY`, `S3_BUCKET`

```

## ## 8. AI: Anthropic Claude

```
Version: `@anthropic-ai/sdk@^0.77.0`
```

Used in two features:

1. **\*\*AI Documents\*\*** (`/ai-documents`) – upload rate confirmations, invoices, and load tenders. Claude reads PDFs and images, extracts structured freight data (origin, destination, rate, dates).
2. **\*\*Profit Predictor\*\*** (`/profit-predictor`) – AI profitability assessment for a potential load, using historical lane and fuel data.

```
- **Required env var:** `ANTHROPIC_API_KEY`
- Model used: `claude-sonnet-4-6`

```

## ## 9. Maps and Geospatial

```
Versions: `leaflet@^1.9.4` · `react-leaflet@^5.0.0` ·
 `react-leaflet-cluster@^4.0.0` · `@turf/bbox@^7.3.4` ·
 `@turf/helpers@^7.3.4`
```

```
- **Leaflet + react-leaflet** – interactive maps in the Owner portal (Live Map, Route Map on load detail page). Must be dynamically imported with
 `ssr: false` in Next.js to avoid SSR issues.
- **react-leaflet-cluster** – clusters nearby truck markers to avoid
 overcrowding the live map.
- **@turf/bbox + @turf/helpers** – geospatial calculations for geofencing
 (detecting when a truck arrives within radius of a stop).
- **`NEXT_PUBLIC_GOOGLE_MAPS_API_KEY`** – enables address autocomplete
 fields using the Google Places API.
```

## ## 10. Charts: Recharts

**\*\*Version:\*\*** `recharts@^2.15.4`

Used throughout dashboards: fuel analytics, safety reports, lane profitability bar chart, profit predictor UI. Some charts use Recharts directly (not via `shadcn/ui ChartContainer`) to support per-bar coloring and multi-field tooltips.

## ## 11. PDF: @react-pdf/renderer

**\*\*Version:\*\*** `@react-pdf/renderer@^4.3.2`

Generates PDF documents server-side:

- **\*\*Invoice PDFs\*\*** – downloadable from the invoice detail page.
- **\*\*Payroll payslips\*\*** – downloadable from the payroll detail page.
- **\*\*Rate confirmation PDFs\*\*** – downloadable from the load detail page.

PDF generation runs in a Server Action (`.tsx` extension required for JSX syntax). Uses the built-in Helvetica font to avoid external font downloads.

## ## 12. Testing

**\*\*Versions:\*\*** `vitest@^4.0.18` · `@playwright/test@^1.58.2`

| Command               | Tool       | Purpose                  |
|-----------------------|------------|--------------------------|
| ---                   | ---        | ---                      |
| `npm test`            | Vitest     | Unit tests               |
| `npm run test:watch`  | Vitest     | Unit tests in watch mode |
| `npm run test:e2e`    | Playwright | End-to-end browser tests |
| `npm run test:e2e:ui` | Playwright | End-to-end tests with UI |



---

## ## 13. Other Utilities

| Package                 | Version   | Purpose                                                                   |
|-------------------------|-----------|---------------------------------------------------------------------------|
| ---                     | ---       | ---                                                                       |
| `bcryptjs`              | `^3.0.3`  | Password hashing for owner/driver login                                   |
| `date-fns`              | `^4.1.0`  | Date formatting and calculations (e.g. document expiry diffs)             |
| `nanoid`                | `^5.1.6`  | Short unique ID generation (email IDs, tracking tokens)                   |
| `sonner`                | `^2.0.7`  | Toast notification system – <code>&lt;Toaster /&gt;</code> in root layout |
| `@tanstack/react-table` | `^8.21.3` | Data tables with sorting and filtering in list views                      |
| `file-type`             | `^13.0.0` | MIME type detection from magic bytes for uploaded files                   |
| `pg`                    | `^8.18.0` | PostgreSQL driver used by Prisma adapter                                  |
| `dotenv`                | `^17.3.1` | Environment variable loading in scripts                                   |
| `@faker-js/faker`       | `^10.3.0` | (dev) Fake data generation for seed scripts                               |

# Feature Modules.md

## # Feature Modules

Reference guide for every major feature module in DriveCommand. Each entry includes the URL path, a description of what the module does, and the key source files.

---

## ## Owner Portal

All owner portal modules live under ``src/app/(owner)/`` and are accessible to users with the ``OWNER`` or ``MANAGER`` role.

---

### ### 1. Trucks — ``/trucks``

Manage the fleet. Add trucks with VIN, make, model, year, and odometer reading. Track truck documents (registration, insurance) with expiry date alerts. View maintenance history and scheduled services per truck.

#### **\*\*Key files:\*\***

- ``src/app/(owner)/trucks/``

---

### ### 2. Drivers — ``/drivers``

Manage drivers within the tenant. Invite new drivers via email (creates a ``DriverInvitation`` record and sends an invitation email with an accept link). View driver profiles, assigned routes, documents, and safety scores. Drivers accept their invitation and set their password via ``/accept-invitation``.

#### **\*\*Key files:\*\***

- ``src/app/(owner)/drivers/``
- ``src/lib/email/send-driver-invitation.ts``

---

### ### 3. Routes - `/routes`

Create and track trips. A Route connects a driver, a truck, an origin, and a destination with a scheduled date. Supports multi-stop routes via the `RouteStop` model (ordered stops with address, type, and arrival status). Owners can track route status (`PLANNED` → `IN\_PROGRESS` → `COMPLETED`) and attach expenses and payments to each route.

#### **\*\*Key files:\*\***

- `src/app/(owner)/routes/`

---

### ### 4. Loads / Dispatch - `/loads`

Manage freight loads through the full dispatch lifecycle: `PENDING` → `DISPATCHED` → `PICKED\_UP` → `IN\_TRANSIT` → `DELIVERED` → `INVOICED`. Loads are linked to customers, drivers, trucks, and routes. Each load gets a unique `trackingToken` that powers the public shipment tracking page at `/track/[token]`.

#### **\*\*Key files:\*\***

- `src/app/(owner)/loads/`
- `src/app/track/` (public tracking page)

---

### ### 5. Invoices - `/invoices`

Create and manage invoices for customers. Invoices can be linked to a load or route. Supports line items, tax, due dates, and status tracking (`DRAFT` → `SENT` → `PAID` → `OVERDUE`). PDF generation for download and email delivery.

#### **\*\*Key files:\*\***

- `src/app/(owner)/invoices/`

---

### ### 6. Payroll - `/payroll`

Track driver pay periods. Each `PayrollRecord` covers a period with base pay, bonuses, deductions, and miles logged. Status lifecycle: `DRAFT` → `APPROVED` → `PAID`. PDF payslip generation for download.

**\*\*Key files:\*\***

- `src/app/(owner)/payroll/``

---

**### 7. CRM — `/crm``**

Customer relationship management. Track shippers and brokers as `Customer` records with contact info, load history, and revenue totals. Log interactions (calls, emails, meetings). Automated interactions are created when loads are updated or ETA notification emails are sent.

**\*\*Key files:\*\***

- `src/app/(owner)/crm/``

---

**### 8. Compliance — `/compliance``**

Document expiry tracking. Shows upcoming expiry dates for truck documents (registration, insurance) and driver documents (licenses, applications). Aggregates safety events per driver over the last 90 days for a compliance score. Triggers email reminders via the daily cron job.

**\*\*Key files:\*\***

- `src/app/(owner)/compliance/``

- `src/lib/email/send-document-expiry-reminder.ts``

- `src/lib/email/send-driver-document-expiry-reminder.ts``

---

**### 9. AI Documents — `/ai-documents``**

Upload and AI-read rate confirmations, invoices, and load tenders. Uses Claude `claude-sonnet-4-6` to extract structured freight data (origin,

destination, rate, dates) from PDF or image uploads. Magic-byte validation runs before the Claude API call. Extracted data can be used to auto-populate load and route forms.

**\*\*Key files:\*\***

- `src/app/(owner)/ai-documents/`
- `src/app/api/documents/`

---

### ### 10. Profit Predictor - `/profit-predictor`

AI-powered profitability analysis for potential loads. Owner inputs a load (origin, destination, rate, weight) and receives an AI assessment: Accept ( $\geq 15\%$  margin), Caution (0-14.9%), or Reject ( $< 0\%$ ). Uses historical lane performance and the tenant's `profitMarginThreshold` setting.

**\*\*Key files:\*\***

- `src/app/(owner)/profit-predictor/`

---

### ### 11. Lane Analytics - `/lane-analytics`

Profitability analysis by route lane (origin-destination pair). Aggregates revenue, expenses, and margin per lane across all completed routes. Displays the top 10 lanes in a bar chart and the full list in a sortable table. Helps owners identify their most and least profitable lanes.

**\*\*Key files:\*\***

- `src/app/(owner)/lane-analytics/`

---

### ### 12. Live Map - `/live-map`

Real-time truck location map. Displays GPS pings from the `GPSLocation` table on a Leaflet map. Truck positions update on a 30-second polling interval. GPS data is submitted by the driver app (browser geolocation) or a connected ELD (Samsara, Motive) via the GPS API endpoint.

**\*\*Key files:\*\***

- `src/app/(owner)/live-map/`
- `src/app/api/gps/`

---

**### 13. Fuel Dashboard - `/fuel`**

Fuel economy tracking. Log fill-ups (quantity, cost, odometer) per truck. Dashboard shows MPG trends, cost per mile, and total fuel spend over time. Supports estimated fuel records for routes without explicit fill-up data.

**\*\*Key files:\*\***

- `src/app/(owner)/fuel/`

---

**### 14. Safety Analytics - `/safety`**

Driver safety scoring. `SafetyEvent` records (harsh braking, speeding, etc.) are logged per truck, driver, and route. Dashboard shows event counts by type and severity, safety scores per driver, and trend charts.

**\*\*Key files:\*\***

- `src/app/(owner)/safety/`

---

**### 15. IFTA - `/ifta`**

International Fuel Tax Agreement reporting. Aggregates miles driven and fuel purchased per jurisdiction using GPS ping data to detect state crossings (bounding-box method). Helps prepare quarterly IFTA filings. CSV export available.

**\*\*Key files:\*\***

- `src/app/(owner)/ifta/`

---

### ### 16. Tags - `/tags`

Color labels for organizing trucks and drivers. Tags are tenant-scoped. Can be applied to trucks or users for filtering and grouping in list views.

#### **\*\*Key files:\*\***

- `src/app/(owner)/tags/`

---

### ### 17. Settings - `/settings`

Tenant configuration. Set tenant name, timezone, and profit margin threshold. Manage third-party integrations (QuickBooks, Samsara, KeepTruckin/Motive, factoring services, email providers). Only accessible to users with the `OWNER` role.

#### **\*\*Key files:\*\***

- `src/app/(owner)/settings/`

---

### ### 18. Support (Owner) - `/support`

Owner submits support tickets to the DriveCommand team. Each ticket has a title, description, category, and priority. Owners and the DriveCommand team can message back and forth through a `TicketMessage` thread. Ticket numbers are auto-generated in `TKT-NNNN` format.

#### **\*\*Key files:\*\***

- `src/app/(owner)/support/`

---

### ## Driver Portal

Driver portal modules live under `src/app/(driver)/` and are accessible to users with the `DRIVER` role.



---

### 19. Driver Portal - `/my-route`, `/my-load`, `/my-tickets`, `/hours`, `/incidents`, `/messages`

Driver-facing interface for active operations:

- **My Route** (`/my-route`) - current assigned route with stop timeline; Mark Departed button for each stop.
- **My Load** (`/my-load`) - current load status with forward-only status advancement buttons (`DISPATCHED` → `PICKED_UP` → `IN_TRANSIT` → `DELIVERED`).
- **My Tickets** (`/my-tickets`) - driver's support tickets and message threads.
- **Hours** (`/hours`) - log Hours of Service (HOS) records.
- **Incidents** (`/incidents`) - report safety incidents with description and severity.
- **Messages** (`/messages`) - in-app messaging.
- **GPS** - browser geolocation tracked in the background and submitted to `/api/gps` on an interval.

**Key files:**

- `src/app/(driver)/`

---

## ## SysAdmin Portal

SysAdmin portal lives under `src/app/(admin)/`. Login is at `/admin/login` using the `ADMIN_SECRET_KEY` - no tenant account required.

---

### 20. SysAdmin Portal - `/admin-dashboard`, `/admin-support`, `/tenants`

DriveCommand internal tools for the platform team:

- **Dashboard** (`/admin-dashboard`) - platform metrics: total tenants, active users, system health.

- **Tenants** (`/tenants`) - view all tenants with status (Pending / Active / Suspended). Create new tenants with owner invitation flow. Activate, suspend, or reactivate tenants.
- **Support** (`/admin-support`) - cross-tenant support ticket queue. Filter by status (Open / In Progress / Closed). Reply to tickets and update status on behalf of the DriveCommand team.

Access requires the `ADMIN_SECRET_KEY` env var to be set. The admin session is managed separately from tenant sessions and uses a cookie named `admin_session`.

**Key files:**

- `src/app/(admin)/`

---

**Shared / Public**

**21. Shipment Tracking - `/track/[token]`**

Public page (no login required) for customers to track their shipment. Accessible via a unique `trackingToken` on each load. Shows GPS map position, status timeline, and estimated delivery info. Polling updates the map every 30 seconds.

**Key files:**

- `src/app/track/`
- `src/app/api/track/`

# Development Setup.md

## # Local Development Setup

A complete guide from zero to a running local instance of DriveCommand. Follow these steps in order.

---

### ## Prerequisites

Before cloning, make sure you have the following installed:

- **Node.js 20+** – [nodejs.org](https://nodejs.org)
- **npm** (bundled with Node.js) or **pnpm**
- **Git**
- A **Supabase account** – [supabase.com](https://supabase.com) (free tier is sufficient). Create a project before the Database Setup step.
- A **Gmail account with a Google App Password** – optional. Required only if you want to test email notifications locally. See the env vars table below for instructions.

---

### ## 1. Clone and Install

```
```bash
git clone <repo-url>
cd drivecommand
npm install
```
```

---

### ## 2. Environment Variables

Copy the example file:

```
```bash
cp .env.example .env.local
```
```

Then fill in the values. The table below documents every variable in ``.env.example``:

| Variable                                       | Required             | Description                                                     | Where to get it                                                                                                                                                                               |
|------------------------------------------------|----------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| --- --- --- ---                                |                      |                                                                 |                                                                                                                                                                                               |
| <code>`DATABASE_URL`</code>                    | Yes                  | PostgreSQL connection string                                    | Supabase Dashboard<br>→ Settings → Database → Connection string (URI). Use port <b>**5432**</b> (direct connection) for local dev. Use port <b>**6543**</b> (Session Mode pooler) for Vercel. |
| <code>`AUTH_SECRET`</code>                     | Yes                  | 32+ character secret for session encryption                     | Run: <code>`openssl rand -base64 32`</code>                                                                                                                                                   |
| <code>`NEXT_PUBLIC_APP_URL`</code>             | Yes                  | App base URL (no trailing slash)                                | <code>`http://localhost:3000`</code> for local dev                                                                                                                                            |
| <code>`RESEND_API_KEY`</code>                  | Email                | Resend API key (legacy email provider)                          | [resend.com/api-keys] ( <a href="https://resend.com/api-keys">https://resend.com/api-keys</a> ) – leave blank if using Gmail SMTP                                                             |
| <code>`RESEND_FROM_EMAIL`</code>               | Email                | Sender address verified in Resend                               | e.g. <code>`DriveCommand &lt;onboarding@resend.dev&gt;`</code>                                                                                                                                |
| <code>`ANTHROPIC_API_KEY`</code>               | AI features          | Enables AI document reading and profit predictor                | [console.anthropic.com/settings/keys] ( <a href="https://console.anthropic.com/settings/keys">https://console.anthropic.com/settings/keys</a> )                                               |
| <code>`S3_ENDPOINT`</code>                     | File uploads         | S3-compatible storage endpoint                                  | Cloudflare R2: <code>`https://&lt;account-id&gt;.r2.cloudflarestorage.com`</code>                                                                                                             |
| <code>`S3_REGION`</code>                       | File uploads         | Storage region ( <code>`auto`</code> for R2)                    | <code>`auto`</code> for Cloudflare R2, or your AWS region                                                                                                                                     |
| <code>`S3_ACCESS_KEY_ID`</code>                | File uploads         | S3 access key ID                                                | Cloudflare Dashboard → R2 → Manage R2 API Tokens                                                                                                                                              |
| <code>`S3_SECRET_ACCESS_KEY`</code>            | File uploads         | S3 secret access key                                            | Same as above                                                                                                                                                                                 |
| <code>`S3_BUCKET`</code>                       | File uploads         | Bucket name                                                     | Your R2 or S3 bucket name                                                                                                                                                                     |
| <code>`NEXT_PUBLIC_GOOGLE_MAPS_API_KEY`</code> | Address autocomplete | Google Maps API key                                             | [console.cloud.google.com] ( <a href="https://console.cloud.google.com">https://console.cloud.google.com</a> ) → APIs & Services → Credentials                                                |
| <code>`ADMIN_SECRET_KEY`</code>                | Yes                  | Password for the sysadmin portal at <code>`/admin/login`</code> | Run: <code>`openssl rand -base64 32`</code>                                                                                                                                                   |
| <code>`CRON_SECRET`</code>                     | Yes (Vercel)         | Protects <code>`/api/cron/*`</code> endpoints                   | Any random string for local testing. Vercel sets this automatically for Vercel Cron Jobs.                                                                                                     |

**\*\*Note on email:\*\*** The ``.env.example`` uses Resend (``RESEND_API_KEY``). The active email implementation in ``src/lib/email/gmail-client.ts`` uses Gmail SMTP instead (requires ``GMAIL_USER`` and ``GMAIL_APP_PASSWORD`` set in ``.env.local`` – not shown in ``.env.example``). See [Email docs](./email.md) for Gmail setup.

**\*\*Minimum required variables for local dev without optional features:\*\***

- ``DATABASE_URL``
- ``AUTH_SECRET``
- ``NEXT_PUBLIC_APP_URL``
- ``ADMIN_SECRET_KEY``
- ``CRON_SECRET``

---

### ## 3. Database Setup

Push the Prisma schema to your Supabase database:

```
```bash
npx prisma db push
```
```

Generate the Prisma client:

```
```bash
npx prisma generate
```
```

The Prisma client is generated to ``src/generated/prisma/`` (not the default location).

**\*\*(Optional) Seed with realistic fake data:\*\***

```
```bash
# Standard seed – owners, drivers, trucks, routes, loads
npm run seed
```

```
# Extended seed - adds fleet intelligence data (fuel records, safety
events, GPS pings)
npm run seed:fleet
```
```

---

## ## 4. Run the Development Server

```
```bash
npm run dev
```
```

The app runs at [http://localhost:3000] (<http://localhost:3000>).

---

## ## 5. Creating the First Admin User

The sysadmin portal is at `/admin/login`. Use the value of `ADMIN_SECRET_KEY` as the password. From the sysadmin portal you can:

- Create a new tenant
- Send an owner invitation email to set up the owner's account

Alternatively, register at `/sign-up` to create an owner account directly, bypassing the invitation flow.

---

## ## 6. Running Tests

```
```bash
# Unit tests (Vitest)
npm test
```

```
# Unit tests in watch mode
npm run test:watch
```

```
# End-to-end tests (Playwright - requires app running)
```

```
npm run test:e2e
```

```
# End-to-end tests with Playwright UI
```

```
npm run test:e2e:ui
```

```
```
```

```

```

## ## 7. Common Issues

### **\*\*"AUTH\_SECRET must be set and at least 32 characters"\*\***

Check that ``AUTH_SECRET`` is set in ``.env.local`` and is at least 32 characters. Generate one with ``openssl rand -base64 32``.

### **\*\*Prisma query errors / RLS violations\*\***

Ensure ``DATABASE_URL`` points to your Supabase project with the correct credentials. For local dev, use the direct connection URL (port 5432). The Session Mode pooler (port 6543) is for Vercel only.

### **\*\*`npx prisma db push` fails with "drift detected"\*\***

This can happen if the schema and database are out of sync. Run ``npx prisma migrate resolve --applied <migration-name>`` to mark the existing state as resolved, then retry ``db push``.

### **\*\*Email not sending\*\***

The active email client requires ``GMAIL_USER`` and ``GMAIL_APP_PASSWORD`` in ``.env.local``. The App Password is a 16-character code from Google's security settings — not your Gmail login password. See [Email docs](./email.md).

### **\*\*`MODULE\_NOT\_FOUND` or import errors after `git pull`\*\***

Run ``npm install`` to install any new dependencies, then ``npx prisma generate`` to regenerate the client if ``prisma/schema.prisma`` changed.



# Vercel Deployment Guide.md

## # Vercel Deployment Guide

How to deploy DriveCommand to Vercel production.

---

### ## Overview

DriveCommand deploys to [Vercel] (<https://vercel.com>). The canonical deploy command is:

```
```bash
npx vercel --prod
```
```

**\*\*Do NOT deploy via GitHub push.\*\*** Always use the Vercel CLI. See project workflow preferences.

---

### ## Prerequisites

- A Vercel account with the DriveCommand project linked
- Vercel CLI installed:

```
```bash
npm i -g vercel
```
```
- All environment variables set in the Vercel Dashboard → Settings → Environment Variables

---

### ## Environment Variables on Vercel

Set every variable from ``.env.example`` in the Vercel dashboard. Key differences from local development:

| Variable | Local value | Vercel value |
|----------|-------------|--------------|
| ---      | ---         | ---          |

```
`DATABASE_URL`	Port 5432 (direct connection)	Port **6543** (Session Mode pooler) – required for serverless
`NEXT_PUBLIC_APP_URL`	`http://localhost:3000`	Your production domain, e.g. `https://drivecommand.vercel.app`
`CRON_SECRET`	Any random string	Vercel sets this automatically – do not set manually
```

#### **\*\*Supabase Session Mode pooler URL format:\*\***

```
...
postgresql://postgres.[ref]:[password]@aws-0-[region].pooler.supabase.com:6543/postgres
...
```

Get this from: Supabase Dashboard → Settings → Database → Connection string → **\*\*Session mode\*\***.

All other variables (`AUTH\_SECRET`, `ADMIN\_SECRET\_KEY`, `RESEND\_API\_KEY`, `ANTHROPIC\_API\_KEY`, S3 vars, etc.) use the same values as local – just add them to the Vercel dashboard.

---

#### **## Build Command**

Defined in `vercel.json`:

```
```json  
{  
  "buildCommand": "node scripts/migrate.mjs && prisma generate && next build"  
}  
...
```

This runs three steps in sequence:

1. ****`node scripts/migrate.mjs`**** – runs SQL migration files atomically. Fails fast with a non-zero exit code if any migration fails (which stops the Vercel build).
2. ****`prisma generate`**** – generates the Prisma client to `src/generated/prisma/`.

3. ****`next build`**** – compiles the Next.js application.

If any step fails, the build fails and the current deployment is not replaced.

Deploying

```
```bash
npx vercel --prod
```
```

This deploys to the ****production URL****. Always use `--prod` – without it, Vercel creates a preview deployment on a separate URL, which does not replace production.

Cron Jobs

Defined in `vercel.json`. Vercel schedules these automatically:

| Path | Schedule | Purpose |
|--|--|--|
| <code>/api/cron/send-reminders`</code>     | Daily at 14:00 UTC ( <code>`0 14 * * *`</code>) | Sends document expiry alerts and maintenance reminder emails to owners |
| <code>/api/warmup`</code>                  | Daily at 08:00 UTC ( <code>`0 8 * * *`</code>) | Keeps serverless functions warm to reduce cold start latency |
| <code>/api/cron/auto-close-tickets`</code> | Daily at 02:00 UTC ( <code>`0 2 * * *`</code>) | Auto-closes support tickets that have been resolved for 7+ days |

Cron routes are protected by checking the ``Authorization: Bearer <CRON_SECRET>`` header. Vercel injects this header automatically when triggering cron jobs.

To trigger a cron manually for testing:

```
```bash
curl -X POST https://your-domain.vercel.app/api/cron/send-reminders \
```

```
-H "Authorization: Bearer <CRON_SECRET>"
```

---

## First Deployment Checklist

- [ ] All env vars set in Vercel Dashboard → Settings → Environment Variables
- [ ] `DATABASE_URL` uses Supabase Session Mode pooler at port 6543 (not 5432)
- [ ] `NEXT_PUBLIC_APP_URL` set to the production domain
- [ ] `ADMIN_SECRET_KEY` set (needed to access `/admin/login`)
- [ ] `AUTH_SECRET` is 32+ characters and consistent with local
- [ ] Email credentials configured (`RESEND_API_KEY` or Gmail vars – see [Email docs](./email.md))
- [ ] R2/S3 bucket created and credentials set (`S3_ENDPOINT`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`, `S3_BUCKET`)
- [ ] `ANTHROPIC_API_KEY` set (if AI features are needed)
- [ ] Run `npx vercel --prod` to deploy

---

## Post-Deployment Verification

After deploying:

1. Visit `https://your-domain.vercel.app` – confirm the landing page loads.
2. Visit `https://your-domain.vercel.app/admin/login` – confirm sysadmin login works with `ADMIN_SECRET_KEY`.
3. Create a tenant and send an owner invitation – confirm the invitation email is delivered.
4. Log into the owner portal – confirm the dashboard loads and database queries return data.
5. Check Vercel Dashboard → Functions → Logs for any runtime errors.

---

## Troubleshooting
```

****Build fails at `node scripts/migrate.mjs`****

The `DATABASE\_URL` may be wrong or unreachable from Vercel's build environment. Confirm the Supabase project is not paused and the connection string is correct.

****502 / Function timeout errors in production****

The serverless function may be hitting cold start latency. The warmup cron at 08:00 UTC mitigates this. If the issue persists, check Vercel's function duration limits.

****"Invalid DATABASE_URL" or Prisma P1000 errors****

Ensure `DATABASE\_URL` uses the Session Mode pooler (port 6543). The direct connection (port 5432) is not supported in Vercel's serverless environment.

Email System.md

Email System

How DriveCommand sends transactional emails, including Gmail SMTP configuration, email templates, all trigger points, and the idempotency system.

Overview

DriveCommand sends transactional emails using **Gmail SMTP** via **Nodemailer**. Email templates are React components rendered to HTML using `@react-email/components` and `@react-email/render`.

Active email client: `src/lib/email/gmail-client.ts`

There is also a legacy `src/lib/email/resend-client.ts` from an earlier Resend-based implementation. It is kept for reference but is not called by any active code.

Gmail SMTP Setup

Required environment variables:

| Variable | Description |
|-----------------------------------|--|
| <code>`GMAIL_USER`</code> | Your Gmail address (e.g. <code>notifications@yourcompany.com</code>) |
| <code>`GMAIL_APP_PASSWORD`</code> | 16-character Google App Password — not your Gmail login password |
| <code>`GMAIL_FROM_NAME`</code> | Optional display name in the From field (defaults to <code>"DriveCommand"</code>) |

Note: These variables are not in `.env.example` (which still lists `RESEND_API_KEY`). Add them manually to `.env.local` for local development and to the Vercel Environment Variables dashboard for production.

How to create a Google App Password:

1. Enable 2-Step Verification on your Google account: Google Account → Security → 2-Step Verification.
2. Go to Google Account → Security → 2-Step Verification → ****App Passwords**** (scroll to the bottom).
3. Create a new app password. Select "Mail" as the app.
4. Copy the 16-character code. Spaces are shown for readability – they are stripped automatically by the client.

How Sending Works

The active client (``src/lib/email/gmail-client.ts``) works as follows:

****`getTransporter()`**** – creates a lazy-initialized Nodemailer transporter singleton:

```
```typescript
nodemailer.createTransport({
 service: 'gmail',
 auth: {
 user: process.env.GMAIL_USER,
 pass: process.env.GMAIL_APP_PASSWORD?.replace(/\s/g, ''),
 },
});
```
```

The transporter is a module-level singleton (``let _transporter``) so it is reused across requests in a warm serverless function.

****`sendEmail({ to, subject, react })`**** – renders the React email component to HTML, then calls ``transporter.sendMail()``:

```
```typescript
const html = await render(options.react);
await transporter.sendMail({ from: FROM_EMAIL, to, subject, html });
return { id: nanoid() }; // compatible with previous Resend-based callers
```
```

The ``FROM_EMAIL`` export is constructed from ``GMAIL_FROM_NAME`` and ``GMAIL_USER``:

- With name: ``"DriveCommand <notifications@gmail.com>"``
- Without name: ``"DriveCommand <notifications@gmail.com>"`` (defaults to "DriveCommand")

Email Templates

Templates are React components in ``src/emails/`` using ``@react-email/components`` primitives: ``Html``, ``Head``, ``Body``, ``Container``, ``Text``, ``Link``, ``Button``, ``Hr``.

Some sender files define their template inline rather than in a separate file. All templates render to HTML before being passed to Nodemailer.

Email Trigger Points

Every email sender function, what triggers it, and what it sends:

| File | Trigger | Description |
|--|--|--|
| --- | --- | --- |
| <code>`src/lib/email/send-driver-invitation.ts`</code> | | Owner invites a new driver from <code>`/drivers`</code> Sends invitation link to the driver's email. Driver clicks link to set password and join the tenant. |
| <code>`src/lib/email/send-owner-invitation.ts`</code> | | SysAdmin creates a tenant from <code>`/tenants`</code> Sends invitation link to the owner's email. Owner clicks link to set password and activate their account. |
| <code>`src/lib/email/send-document-expiry-reminder.ts`</code> | Daily cron job (<code>`/api/cron/send-reminders`</code>) | Alerts the owner when a truck document (registration, insurance) expiry is approaching (at 90/60/30/0-day milestones). |
| <code>`src/lib/email/send-driver-document-expiry-reminder.ts`</code> | Daily cron job (<code>`/api/cron/send-reminders`</code>) | Alerts the owner when a driver document (license, application) expiry is approaching (at 90/60/30/0-day milestones). |

```
| `src/lib/email/send-maintenance-reminder.ts` | Daily cron job  
(`/api/cron/send-reminders`) | Alerts the owner when a scheduled truck  
maintenance service is due. |  
| `src/lib/email/send-geofence-alert.ts` | GPS geofence trigger after each  
GPS ping | Notifies the customer when a truck arrives within the geofence  
radius of a pickup or delivery location. |  
| `src/lib/email/customer-notifications.ts` | Load status changes  
(`PICKED_UP`, `IN_TRANSIT`, `DELIVERED`) | Sends ETA and status update  
emails to the customer linked to the load. Fire-and-forget – email failure  
never blocks the load status update. |  
| `src/lib/email/send-support-notifications.ts` | Support ticket activity  
| Notifies the DriveCommand team (via `DRIVECOMMAND_SUPPORT_EMAIL` or  
`GMAIL_USER`) when a new ticket is created; notifies the owner when the  
admin posts a reply. |
```

Notification Log (Idempotency)

Every email send is recorded in the `NotificationLog` database table with a unique `idempotencyKey`. Before sending, the system checks whether a log entry already exists for that key. If it does, the email is skipped.

This prevents duplicate emails from:

- Cron job retries after a transient error
- Double form submissions
- Rapid repeated triggers

Pattern used in cron email senders:

```
```typescript  
const existing = await prisma.notificationLog.findUnique({
 where: { idempotencyKey: key }
});
if (existing?.status === 'SENT') return; // skip – already sent

// ... send email ...

await prisma.notificationLog.create({
 data: { idempotencyKey: key, type: 'document-expiry', status: 'SENT' }
});
```

```
});
```
```

For document expiry reminders, the idempotency key includes the document ID and the milestone threshold (e.g. ``doc-expiry-truck-abc123-30days``) so each milestone is tracked independently.

Testing Email Locally

Set ``GMAIL_USER`` and ``GMAIL_APP_PASSWORD`` in ``.env.local``. Emails will be sent from your Gmail account during local development – there is no sandbox mode.

****Recommendation:**** Use a test email address (e.g. a Gmail alias or a secondary account) as the recipient during development to avoid cluttering real inboxes.

To trigger the cron email job locally:

```
```bash  
Start the dev server, then:
curl -X POST http://localhost:3000/api/cron/send-reminders \
 -H "Authorization: Bearer <your-CRON_SECRET-value>"
```
```